



IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chemical Mathematical Institute
Indian Institute of Technology Madras
Abstract Classes and Interfaces

(Refer Slide Time: 00:13)

Abstract classes and interfaces

Madhavan Mukund
<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java
Week 4

Grouping together classes

- Sometimes we collect together classes under a common heading
- Classes `Circle`, `Square` and `Rectangle` are all shapes
- Create a class `Shape` so that `Circle`, `Square` and `Rectangle` extend `Shape`
- We want to force every `Shape` to define a function `public double perimeter()`
- Could define a function in `Shape` that returns an absurd value `public double perimeter() return -1.0;`
- Rely on the subclass to redefine this function
- What if this doesn't happen?
 - Should not depend on programmer discipline

So, we have seen the Java class hierarchy. So, now we will talk about some new concepts involving classes, namely, Abstract classes and interfaces. So, the class hierarchy is essentially a way to group together objects of different types into a common hole. So, for instance, we might have classes which represent geometric shapes like circle, square and rectangle, and to tie these together, we might create a parent class called shape, so that, all these three, extend that right. So, we could have a hierarchy in which we have shapes, and then we have below that we have circle, rectangle, and square.

So, suppose we want every shape to have some particular function implemented. For example, perimeter, we might want to define perimeter in every shape. Now, notice that the definition of perimeter varies from shape to shape. So, if it is a circle, it is two times five times the radius, if it is a square, it is four times the length to the side. If it is the rectangle, you have a length and a width, and it is two times the sum of the length. So, depending on the actual shape, the definition of perimeter varies, but you want to make sure that all these subclasses of shape definitely have a definition of perimeter.

Now, since this definition varies, it is also not simple to define a default perimeter function, which makes sense for all of these because you must actually make sure that the perimeter that is calculated by shape is sensible. So, one way to do this is to create an abstract definition inside shape, which is kind of vacuous.

So, you could have a function which is called perimeter. So, let us just write some brackets which are missing. So, you could have some function inside shape, which is called perimeter and it returns a negative number. Now, clearly the perimeter of a shape cannot be negative, so you know that this is wrong. So, what it means is that you if you see a minus 1, you know that the class was written, the subclass were written improperly, so you are actually relying on the programmer of the subclass to redefine this function.

Now, we have seen before that, this is something that object-oriented programming in general tries to address. If you have discipline programmers, then many of the features that we have introduced like public and private encapsulation, and all that need not actually be used, because you can hope that programmers who are disciplined will not exploit the internal structure, they will obey the interface and so on.

But the problem arises because typically, software is written by groups of people across time with different coding styles and different coding practices, and you cannot really, and you should not really depend on programmer discipline. So, we would like to have something like this, which is enforced, independent of whether a programmer chooses to do so or not.

(Refer Slide Time: 03:15)

Abstract classes

- A better solution
 - Provide an **abstract** definition in **Shape**
`public abstract double perimeter();`
 - Forces subclasses to provide a concrete implementation
 - Cannot create objects from a class that has abstract functions
 - **Shape** must itself be declared to be **abstract**

```
public abstract class Shape{  
    ...  
    public abstract double perimeter();  
    ...  
}
```

new Shape()

So, a better way of doing this is to create what is called an abstract definition. So, you are creating just like a class is a template in itself, within a class, you are creating a template for a function, you are providing a signature. You are providing a signature for a function, but you are not providing the body of the function.

So, you are just promising that there is a function called parameter whose return value is double, which does not take any arguments, but you are saying that I am not providing anything for it, and that you are saying by using this key word that Java uses called abstract. So, an abstract function definition is a function header.

If you work in a language like C or C++, then very often you use these kinds of function headers up front at the top of your code to define functions, which exist outside, so that if you call a function inside this file, then the compiler knows that it matches the signature and it looks for another function.

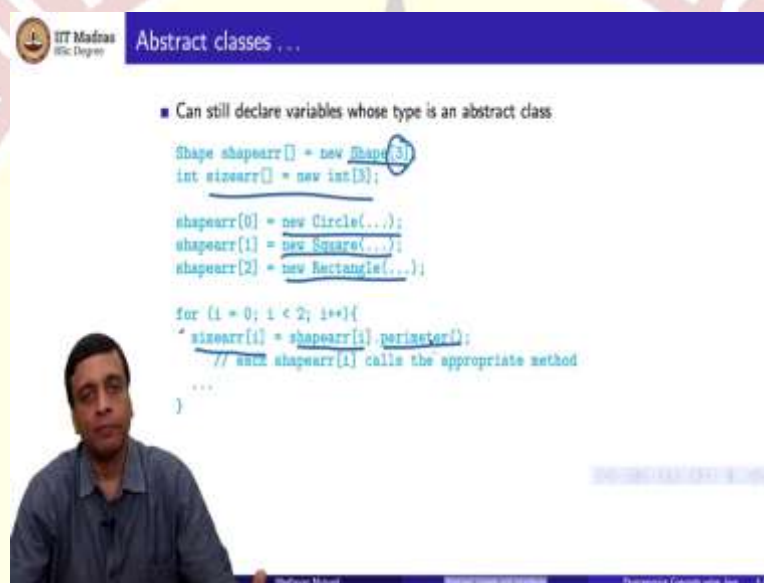
Now, this is a different use of abstract here. So, what we are saying, is that, this class requires this function to be implemented by any subclass that has it because this one does not have a definition. So, every subclass is now forced to provide a concrete implementation of this abstract class.

Now, clearly if I have a function, which is not defined, then I cannot create an object of that type, because this now this object will come equipped with a function which I cannot execute. I know its signature, but there is no code to execute it. So, in such a situation, the

class itself has to be abstract. So, if you have an abstract method inside your class, then the entire class has to be called abstract. So, this is a syntactic requirement in Java.

If you put an abstract method in a class. There could be other methods which are not abstract. But if you put an abstract method, any abstract method into a class, the entire class has to be declared to be abstract otherwise, it is an error. So, once you have an abstract class, as we said, you cannot create objects. So, we cannot make, we cannot say new shapes. Because if we create a new shape, then what would we use for the perimeter of that object? We do not have code for that.

(Refer Slide Time: 05:28)



Abstract classes ...

- Can still declare variables whose type is an abstract class

```
Shape shapearr[] = new Shape[3];
int sizearr[] = new int[3];

shapearr[0] = new Circle(...);
shapearr[1] = new Square(...);
shapearr[2] = new Rectangle(...);

for (i = 0; i < 3; i++){
    sizearr[i] = shapearr[i].perimeter();
    // since shapearr[i] calls the appropriate method
    ...
}
```

But you can create variables whose type is abstract. So, remember that we use this kind of subclassing thing to create this heterogeneous collection. So, we had in the original simula, we had a queue of different types of events, all of which can be put together in a single object and be well typed. And we saw in the case of subtyping, for example, that we could have an array of employees and some of the employee objects in that array could be employees, and some of them could be managers, which is a subtype.

So, in that same sense, we could create an array of shapes, but we are not going to populate this array with actual shapes, because we are not allowed to create shapes because shapes are abstract. So instead, what we can say is that say we have got a three-element array of shapes, we can say that the first element the 0 index, is a circle, which is a concrete object, and it presumably has because of this requirement, that perimeter has to be concretely defined in every subclass it, if we can create a new circle, it must have defined it.

Now, it could postpone it. Supposing circle also keeps it abstract, then you cannot create circles, you have to do something else. Maybe we have another subtype of circle. But if we can create a circle, it means that circle extends shape, and circle has an implementation of the subtract structure. Similarly, the second element in our shape array is a square, and the third element is a rectangle.

So, we have a placeholder called shape, which accommodates all these different types of shapes, and we can now glue them together in a single array whose overall type of shape but whose individual components are these concrete objects have a subtype of shape. So, this is allowed. And this is what makes this idea of having these abstract classes useful, because you can still think of these as all being different objects of a parent type, and all having common capabilities. But now, of course, these are all new dynamic dispatch going to use their own thing.

So, supposing I run through a loop and I accumulate in this other array, the sizes of each of these shapes in terms of their perimeter, then I will look at the *i*th element in shape, and compute its perimeter. But now the definition of perimeter will be different, as we said for circle, different for square and different for rectangle, and these will automatically be called depending on the runtime value of that object. So, using dynamic dispatch, I can have this kind of simple loop which calculates the perimeter of every object without worrying about which particular object it is, because dynamic dispatch will make sure that the correct parameter function is called.

(Refer Slide Time: 08:08)

IIT Madras
Public Degree

Generic functions

- Use abstract classes to specify generic properties

```
public abstract class Comparable{  
    public abstract int cmp(Comparable a);  
    // return -1 if this < a,  
    //      0 if this == a,  
    //      +1 if this > a  
}
```

Handwritten diagram illustrating comparison logic:

mu

A < B
A == B
A > B

Small inset video of the lecturer in the bottom left corner.

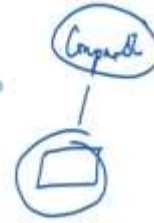


- Use abstract classes to specify generic properties

```
public abstract class Comparable{
    public abstract int cmp(Comparable a);
    // return -1 if this < a,
    //      0 if this == a,
    //      +1 if this > a
}
```

- Now we can sort any array of objects that extend Comparable

```
public class SortFunctions{
    public static void quicksort(Comparable[] a){
        ...
        // Usual code for quicksort, except that
        // to compare a[i] and a[j] we use a[i].cmp(a[j])
    }
}
```



So, we can extend this idea a little further. So, we can now use these abstract capabilities to define some kind of generic properties. So, remember that we talked about sorting. So, what do we want to do when we sort an array, we want to take any two elements in the array and compare them. We want to know whether one is bigger than the other or not. So, there are three possible outcomes of a comparison.

So, supposing I have A and I have B. When I compare A to B, A could be strictly smaller than B, A could be equal to B or A could be strictly greater than B. So, there is a standard convention for this kind of a comparator. A function, which tells us whether it is smaller equal to a bigger, which returns minus 1. So, if A is less than B, you return minus 1 if A is equal to B, you return 0 and if A is greater than B you return plus 1.

Now, in the object-oriented context, A is always this. I am doing it in the context of one object and this is the other one. So, I will pass a compatible object S to something which is here, and I will check whether this object is smaller than that object. This object is equal to that object or this object is greater. If this object is smaller a return of minus 1, if it is equal to a return is 0. If it is greater than return a plus 1, but notice that this is abstract.

I have not told you anything. So actually, you can violate this also, but there should be some kind of a comparator function with this signature. Now, if you have this requirement, then anything which implements which sort of extends, sorry, this class comparable will now be sortable.

So, if I can take any array, which is of type comparable. So, remember, if I have comparable and I have something below it, then this array of objects of the subtype can be used in an

array of the bigger type. So, I say comparable of A, and then I can compare ai and aj, because I know that the cmp function is there. So, I can use ai dot cmp aj in order to find all the ai is bigger than ai, ai smaller, and this is essentially what a function like quicksort would use.

(Refer Slide Time: 10:30)

The top screenshot shows a slide titled "Generic functions ..." with the following Java code:

```
public class SortFunctions{
    public static void quicksort(Comparable[] a){
        ...
    }
}

■ To use this definition of quicksort, we write

public class Myclass extends Comparable{
    private double size; // quantity used for comparison

    public int cmp(Comparable a){
        if (a instanceof Myclass){
            // compare this.size and ((Myclass) a).size
            // Note the cast to access a.size
        }
    }
}
```

The bottom screenshot shows the same slide without annotations.

So, we have this kind of a function abstract quicksort in our class, for example, so this is a static function and it compares, it takes an array of comparable elements. Now, in order to use this, I must provide my type with this capability. So, if I have my own class, then it must extend this comparable abstract type that we have just defined abstract class. And what we need to do in our class is to implement this function comparable. So, we need this this abstract function cmp, which is required by this abstract class has to have a concrete implementation in Myclass.

So, this is a little bit like the equals that we had used earlier for object. So, we take this input instance S, and check whether it is also an instance of the same class. I mean, I can only compare two objects if they are both of the same type. So, I check whether the incoming object to the cmp function is the same type.

If it is of the same type. For example, I might have some quantity called size or something which represents the size of this object, so then, I will compare my size this dot size with S dot size, but I cannot just do S dot size, because S comes with this abstract notion of compatibility. So, I have to first convert S to be compatible with Mytype. So, I have to first use this casting to make S into my class.

Then say, the cast version of S, the Myclass version of S, what is it's size what is my size, and depending on that I return minus 1, 0 or plus 1. So, there are two parts this. On the one side, somebody defines a sort function, which expects a certain capability of all the objects. So, the array that is passed all extends this abstract class comparable.

And when we use it, we have to make sure that the type that we want to sort using that function also extends comparable and therefore in particular, it must implement that cmp function, which that abstract class has made abstract with a kind of requirement that it must be present in order for quicksort to work. So, this is how this whole abstract class idea.

So, the first thing we talked about abstract classes was that it can enforce a certain discipline within a group of objects. So, you want to make sure that all shapes have a perimeter, and a valid perimeter function. So, you define a parent function which has no body and make sure that every shape is forced to implement it in some sensible way. And hopefully, if the person is required to implement it, they will put a sensible implementation. But at the very least, they cannot get away by not implementing it and relying in default thing.

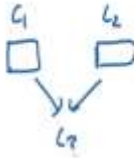
But now you are saying that not only can it require something to be happen, it can also be the other way around can say that there is a certain capability that any subtype of this has. And if that capability is coded correctly, then you can use that capability in another function, so the sort function can use this compatibility.

(Refer Slide Time: 13:33)

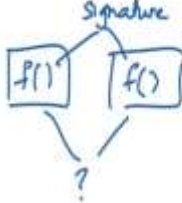
IT Madras
BSc Degree

Multiple inheritance

- Can we sort `Circle` objects using the generic functions in `SortFunctions`?
 - `Circle` already extends `Shape`
 - Java does not allow `Circle` to also extend `Comparable`!



Signature

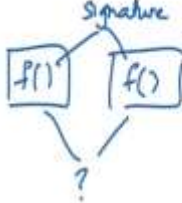


IT Madras
BSc Degree

Multiple inheritance

- Can we sort `Circle` objects using the generic functions in `SortFunctions`?
 - `Circle` already extends `Shape`
 - Java does not allow `Circle` to also extend `Comparable`!
- An interface is an abstract class with no concrete components

```
public interface Comparable{  
    public abstract int cmp(Comparable a);  
}
```



IT Madras
BSc Degree

Multiple inheritance

- Can we sort `Circle` objects using the generic functions in `SortFunctions`?
 - `Circle` already extends `Shape`
 - Java does not allow `Circle` to also extend `Comparable`!
- An interface is an abstract class with no concrete components
- A class that extends an interface is said to **implement** it:

```
public class Circle extends Shape implements Comparable{  
    public double perimeter(){...}  
    public int cmp(Comparable a){...}  
    ...  
}
```

Can extend only one class, but can implement multiple interfaces

So, now we come into this problem of multiple inheritance. So, we know that we can group together our shapes under a subtype shape as subtypes of shapes, so we can have circles, squares and rectangles, and we know that we can group them as having this capability of being comparable. But supposing I take a circle now, an array of circles, and I want to sort it using that generic quicksort. The difficulty is that circle already extends this abstract class shape, but it also needs to extend the other abstract class comparable. And Java does not allow this.

At no time does Java allow two classes to feed into another class. So, I cannot have C1, C2, whether they are abstract or concrete, and have C3, which inherits from both of them. So, how do we get around this? So, for this Java actually comes up with a slightly refined notion of an abstract class. So, this is something called an interface. So, remember that in an abstract class, the reason that a class becomes abstract, is that, at least one of the methods defined in that class is abstract.

The moment you have some component of a class, which is not concretely defined, the entire class becomes abstract. So, you could have an abstract class which has some concrete components and some abstract components. It has at least one abstract component normally, but it could have other parts which are fully defined. But supposing you have nothing which is defined, it is only a set of these templates which are desired from the child class.

So, if you have only abstract components in your class, it is still an abstract class, but it has a special property in Java, it is called an interface. Now, the advantage of an interface, is that, remember, the reason why we did not want multiple inheritance in Java, if you remember is that if I had an implementation of f, in two parent classes, and I wanted to know what would be the implementation of f that I should use in the child class. But if I have an interface, I have a specification. So, both these fs will have the same signature, that is for sure, but there is no implementation.

The interface does not provide you is guaranteed to be abstract. So, an interface is an abstract class in which no concrete components are there. So here, for instance, if I just have this one abstract signature of the cmp function, then I can put it into a class and call it not a class but an interface. So, this abstract class is substituted by a different syntactic word interface. And now if I have an interface, it is guaranteed not to provide a contradictory implementation.

So, the problem is with contradictory implementations. I derive two different, I inherit two different implementations. I do not know which to execute, so I am forced to override it or

get confused. Here, there is no confusion because I have to implement it because it is coming abstractly from somewhere, so there is no problem. So, when we extend an interface the terminology changes to implements.

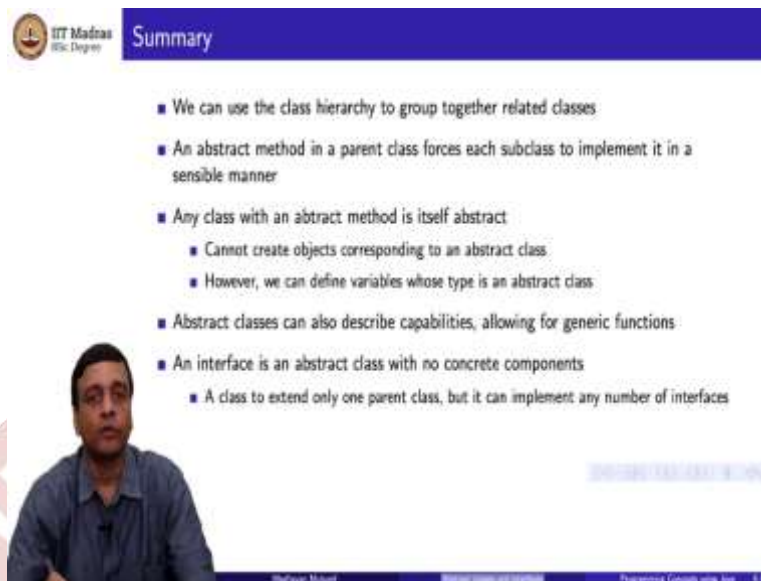
So, what Java says is that you can say that the class circle extends shape, as it did before, but now I have converted my comparable to just an interface. It has only one abstract definition as its component, it has no concrete components. So, it implements comparable. So, there are, so this is again something that the compiler can check that you extend only one thing but you can implement now more than one thing that seems interesting.

Now, of course, implementation of an abstract interface is the same as implementing an abstract class, you still have to provide a definition for that you cannot get away with that. If you claim to implement an interface, it has some abstract components and you must actually implement them, you must provide a concrete implementation. Similarly, this abstract class shape had its own requirement because it had an abstract parameter and you are forced to implement that.

So, the requirement for the subclass remains the same. Everything which comes down to it as an abstract function, abstract method it must implement, but the ones which are purely abstract in terms of interfaces cannot syntactically create any confusion because they cannot provide any rival implementation for a function, and therefore, you can safely inherit from multiple sites.

So, you can extend only one class whether it is concrete or abstract does not matter, but you can implement as many interfaces as you want, because you are guaranteed that every interface is purely abstract. It has only abstract components, nothing concrete, so there can be no contradiction in the implementation.

(Refer Slide Time: 18:12)



Summary

- We can use the class hierarchy to group together related classes
- An abstract method in a parent class forces each subclass to implement it in a sensible manner
- Any class with an abstract method is itself abstract
 - Cannot create objects corresponding to an abstract class
 - However, we can define variables whose type is an abstract class
- Abstract classes can also describe capabilities, allowing for generic functions
- An interface is an abstract class with no concrete components
 - A class to extend only one parent class, but it can implement any number of interfaces

So, to summarize, what we have seen is that the class hierarchy helps us to group together classes. So, we saw that we can take circles, shapes, rectangles, and squares and group to group them together as a shape. And now if we want a capability to be properly implemented in all these, we can create an abstract function inside the parent class.

And this abstract function, since it has no body will require the subclasses to actually implement these in a sensible way. But when we do this, the parent class also becomes abstract and we cannot create objects of that type, but we can create variables of that type. So, we can have these kinds of containers like arrays of shapes, which can have different concrete objects inside them.

The other thing we saw is, that we can extend this idea of what abstract functions to describe capabilities like compatibility. So, any function, any class which implements a compare function properly, can be sorted. And in order to now allow these multiple capabilities to coexist in a class to be able to signal that this class can do capability 1, capability 2, capability 3 so it can do various things. If these are all abstract capabilities, then we can label them not as abstract classes but as interfaces. And in that setting, a class can implement multiple interfaces, though it can only extend one class.